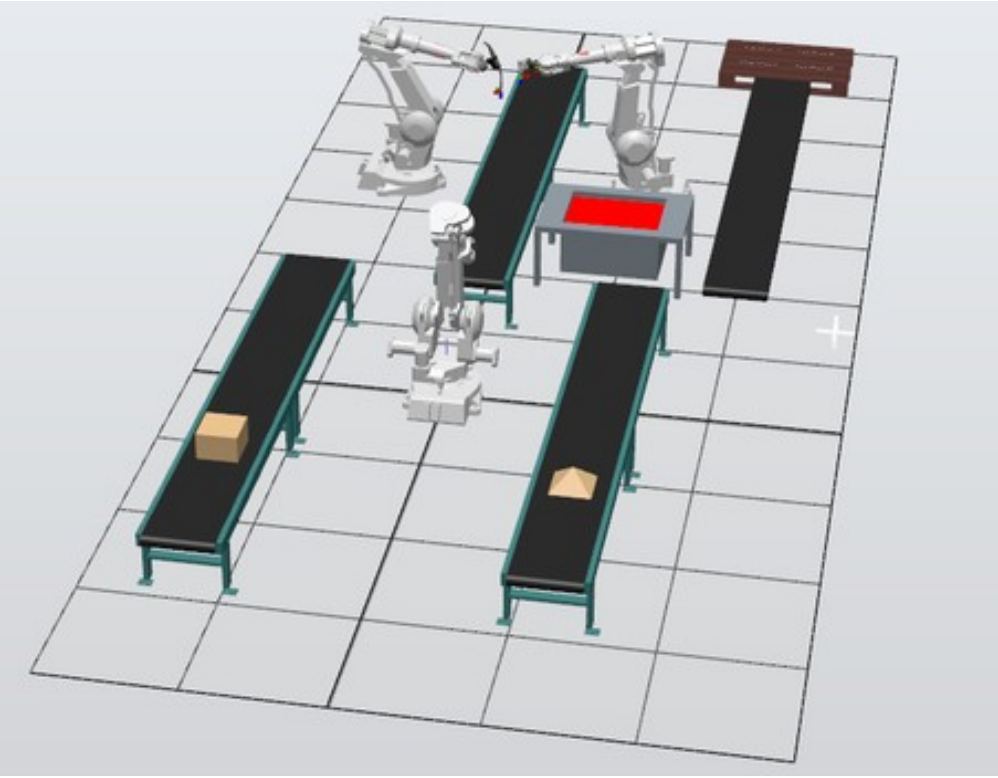
ENGINEERING PROJECT REPORT

SMART FACTORY DESIGN & SIMULATION

A virtual four-stage manufacturing line developed in ABB RobotStudio



Station overview reproduced from the team's original one-page report [R1].

Rémi Dalmasso • Ethan Diai • Hippolyte Farion • Damien Perrier

Course: Introduction to Smart Factory & Digital Manufacturing

Supervisor: Sihao Deng | Academic year: 2026

Final project report • Introduction to Smart Factory & Digital Manufacturing • 2026.

REPORT OVERVIEW

Abstract

Purpose, technical approach and main findings.

The objective of this project was to conceive a smart manufacturing scenario and represent it as a virtual production line with ABB RobotStudio. The chosen example combines two initially independent components into a single product and then routes that product through four successive operations: pick and place, welding, painting and packaging. The model was intended to make the relationships between robotic manipulation, simulated transport and logic-based station control visible and testable before any physical implementation.

The technical approach relies on RobotStudio Smart Components. Sources create parts; LinearMovers provide the movement associated with conveyors; PlaneSensors detect arrival events; Controller and signal links organize the transitions; Attachers and Detachers manage simulated object association; and the Sink removes objects from the station. This division of responsibilities is the central modelling decision evidenced by the original report. The team also produced a visual representation of the station and documented the functional chain in a separate presentation.

Three practical limitations were reported by the team: simulation instability when the computer lacked sufficient resources, inconsistent results with apparently unchanged logic, and downstream effects when software errors accumulated. These observations are important because a production scenario can be logically convincing while remaining difficult to validate reproducibly. The available documents do not provide validated cycle-time measurements, repeat-run statistics, exact robot model identifiers or the complete RAPID programs; the present report does not manufacture these missing details.

Project element	Documented outcome
Scenario	Four successive manufacturing operations
Digital representation	RobotStudio station overview and described logic
Control approach	Smart Components connected by event signals
Main limitations	Stability, repeatability and error propagation
Next validation priority	Reproducible end-to-end simulation with a logged test method

NAVIGATION

Contents and reference documents

Section	Pages (approx.)	Subject
1. Introduction	4	Objectives and project deliverables
2. Context and scope	5–6	Industrial purpose, assumptions and boundaries
3. Technical specifications	7–15	Production line, robot and station logic
4. Problems and constraints	16–17	Stability, repeatability, cumulative errors
5. Solutions and design choices	18–19	Modular logic and suggested controls
6. Implementation process	20–23	Modelling, integration, debugging
7. Results and discussion	24–26	Established outcomes, interpretation, tests
8. Conclusions and perspectives	27–28	Lessons learned and development roadmap

Evidence convention

The report distinguishes the virtual station and control principles developed during the project from the tests and improvements proposed for future work. Where quantitative measurements were not available, the analysis remains qualitative.

Source cross-reference

[R1] Dalmasso, R., Diai, E., Perrier, D., and Farion, H. (2026). “Smart Factory Design & Simulation.” Initial group project report. [R2] Dalmasso, R., Diai, E., Farion, H., and Perrier, D. (2026). “Smart Factory Design & Simulation: A Digital Twin of an Automated Production Line in ABB RobotStudio.” Group project presentation.

In the production scenario, painting is performed by immersion in a tray of paint, as described in the initial project report [R1].

1. Introduction: problem statement and objectives

Documented in the assignment overview and the team's stated project goal [R1, R2].

Modern automated lines require more than the movement of a single robot. Several units must operate in the right order, parts must arrive at the expected locations, and each transformation must be triggered at an appropriate moment. A virtual factory offers a way to model such interactions before investing in physical machines. For this assignment, the team selected a deliberately understandable manufacturing case rather than a complex industrial product.

The proposed process begins with two separate components travelling on different input conveyors. An articulated arm takes them and superposes them on a receiving conveyor. They are then treated as an assembled product at a welding stage, pass through a painting stage and are finally placed in a box. The intended transformation can be expressed as two input components leading to one finished, coated and packaged product.

Expected deliverables defined by the coursework

1. Define a coherent Smart Factory production scenario with a sequence of operations.
2. Create a virtual station and corresponding objects in ABB RobotStudio.
3. Develop the robot programs and configure the behaviour of the simulated components.
4. Simulate the production sequence and evaluate whether the operations interact correctly.

This report presents the project objectives, the design of the production sequence, the technical control architecture, the problems encountered during simulation, the solutions selected, and an assessment of the outcomes and future development opportunities.

2 • CONTEXT

2.1 Why a virtual factory was relevant

Engineering context and intended benefits, not claims about a commissioned plant.

An automated production line is a sequence of dependent processes. In a real factory, a late or misplaced component may cause a robot to wait, an operation to start at the wrong time, or an accumulation of parts between stations. A simulation helps the designer inspect these dependencies in a controlled visual environment. It also makes it possible to examine the influence of virtual sensors and signal wiring without involving physical machinery.

The RobotStudio environment was chosen because the coursework required its use for station creation and robotic simulation. The team used the 3D environment to position the elements of the line and Smart Components to represent their associated behaviour. A sensor indicates the presence of a component; the control logic then determines what should occur next. In this setting, the value of the model lies mainly in the coordination between distinct operations.

Project context: educational model rather than industrial certification

Potential digital-factory benefit	What this project actually demonstrates
Visualising a future line	A visible 3D layout with robot and conveyor elements
Representing material flow	Parts created and moved through a planned sequence
Testing event coordination	PlaneSensors and connected Smart Components described
Checking production capacity	Not quantified in the current evidence
Validating physical process quality	Not established for actual welding, paint or packing

This scope matters for the conclusion. The model is a valid exercise in manufacturing-system architecture, but an industrial deployment would additionally demand mechanical feasibility, controls safety, cycle-time consistency, process-quality requirements and repeatable validation.

2 • CONTEXT

2.2 Scope, system boundary and working assumptions

The assumptions below are used to interpret the supplied project, not presented as measured values.

Boundary of the studied system

The input boundary consists of two virtual components generated at the beginning of the line. The process boundary includes the assembly robot, transport paths, welding stage, painting stage and packaging stage. The output boundary is a completed object reaching the final point of the model, where the Sink may remove it from the simulation. The virtual system is therefore concerned with the transformation and movement of digital objects.

The source documents do not specify part geometry and material, robot payload, conveyor speed, workcell dimensions, actuator properties, tool models or electrical I/O addressing. These missing items should not be silently filled with assumptions: they prevent a quantitative feasibility study from being completed.

Documented boundary versus missing design data

Topic	Source-supported information	Information still required
Product	Two components, one product	CAD drawings, material, mass, tolerance
Robots	Articulated arm(s) in station	Exact ABB model, tool, payload, reach
Transport	Conveyors represented by LinearMovers	Lengths, speeds, buffer capacities
Join/finish	Weld + coating + packaging stages	Process timings and quality criteria
Control	Event-based Smart Components	Signal map, safety interlocks, code export
Performance	Problems with unreliable execution	Cycle-time trials, rates, error statistics

The station illustration is a visual source, not a dimensioned general-arrangement drawing; robot counts and mechanical capacities should not be inferred from it alone.

3 • TECHNICAL SPECIFICATIONS

3.1 Overall production sequence

Functional specification of the selected four-operation scenario [R1, R2].

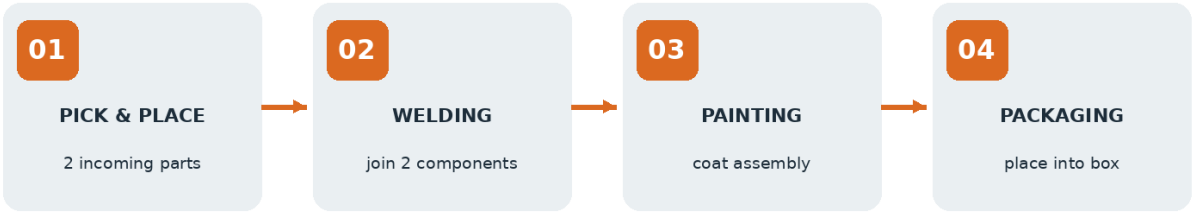


Figure 1. Functional manufacturing sequence, redrawn from the project documents [R1, R2].

The process is sequential: downstream steps require a product to have been supplied by earlier ones. The design is built around the following logic. Two independently generated components first arrive at an assembly area. After their relative placement is established, they are considered a single workpiece for the rest of the sequence. The welded assembly then receives a surface treatment and is finally transferred to a box.

Stage	Input	Expected transformation	Output
1 / Pick & place	Part A + Part B	Robot picks and superposes parts	Aligned pair
2 / Welding	Aligned pair	Modelled joining operation	Joined assembly
3 / Painting	Joined assembly	Assembly is immersed in a paint bath	Coated assembly
4 / Packaging	Coated assembly + box	Product is placed into a box	Packaged product

The sequence defines the principal functional requirement: an operation must not start until the right object is present. Presence is inferred through sensors and signal events. This makes the station logic essential for the coherence of the simulated production cycle.

3 • TECHNICAL SPECIFICATIONS

3.2 Virtual layout and spatial organisation

Evidence: isometric RobotStudio screenshot reproduced in the initial report [R1].

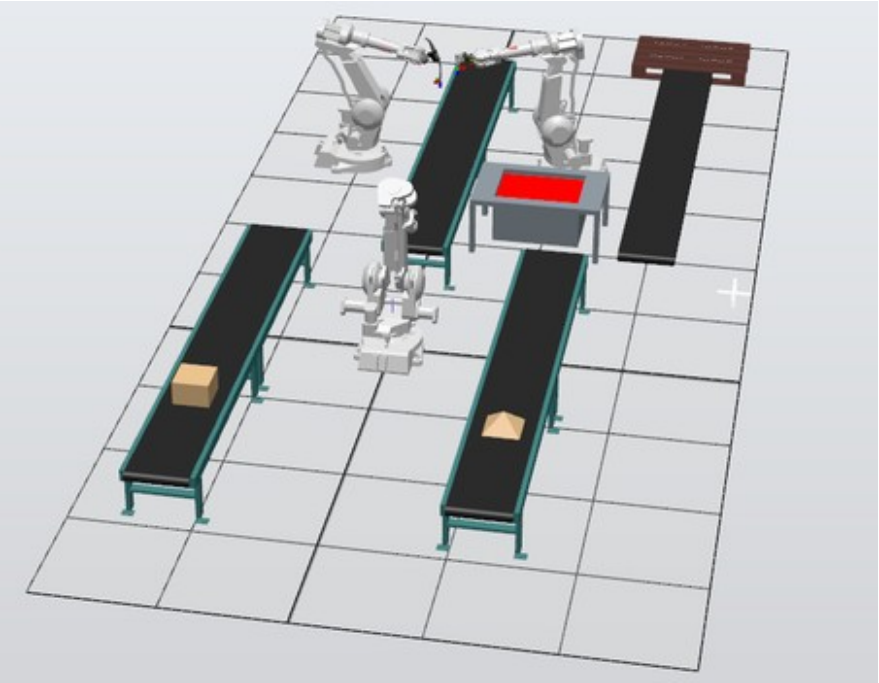


Figure 2. Original RobotStudio overview captured in the one-page team report [R1].

The available station image shows a model composed of robotic devices, dark conveyor-like paths, small transported parts and a central work area. The scene represents a common reference environment in which the operations are intended to interact. The screenshot is useful as proof that the production scenario was modelled visually; however, it does not establish reach limits, tool kinematics, collision-free paths or the exact mechanical specifications of each component.

Why the arrangement matters

Spatial placement defines whether a robot can physically reach the pick position and delivery target, whether the product travels in the intended direction and whether sensors are positioned before a control action must occur. Even in a simplified model, the transport path and the target frames should refer to consistent coordinates so that object detection and robot motion describe the same production event.

For a detailed layout review, the relevant positions are the two input conveyors, the assembly conveyor, the finishing areas, the packaging location and the detection sensors. Their precise coordinates belong to the RobotStudio station model and cannot be established from the overview image alone.

3 • TECHNICAL SPECIFICATIONS

3.3 Pick-and-place cell: required behaviour

Detailed decomposition of the first operation from the documented scenario [R1, R2].

Purpose and sequence

The assembly cell takes one component from each of two separate material streams and superposes them on a third conveyor. This operation is the main convergence point of the line: two independent objects must become a correctly positioned pair. The cell therefore depends on both robot motion and the timing of the simulated conveyor flows.

5. A Source introduces the first component at the first infeed location, and another source provides the second component.
6. A LinearMover transports each component along the virtual conveyor path.
7. A PlaneSensor detects a component in its relevant arrival area and makes its presence visible to the station logic.
8. The robot must be allowed to pick only when the appropriate part is at the intended location.
9. The two parts are superposed at the assembly target so the next station can act on them.

Critical interfaces

Interface	Main design question	Evidence
Infeed → detection	Is each part detected at the right target?	Sensors described; positions not listed
Detection → robot	Which signal releases the motion?	Signal approach described; wiring not supplied
Robot → assembly	Are the two target poses consistent?	Operation described; paths not supplied
Assembly → transport	When may the paired object move?	Attacher and LinearMover logic described

The central risk is premature or delayed transfer. A sensor event without a corresponding completed robot action could move an unassembled part downstream; an isolated robot action without confirmation of arrival could attempt a pick from an empty position.

3.4 Welding, painting and packaging stages

The coating stage uses immersion in a tray of paint.

Welding

Following the positioning of the components, the intended welding operation joins them into one part. In the simulation logic, Attacher provides a way to associate digital objects so that they move as a combined assembly. This is a modelling representation of joining; it is not proof of a realistic welding simulation with material change, heat input, bead geometry or quality assessment. No welding program or process parameter set is documented in the supplied files.

Painting by immersion in a paint bath

After the two components have been positioned and joined, the assembly is dipped into a tray of paint. This immersion method is the confirmed painting operation for the project and agrees with the initial report [R1]. The robot or handling system must bring the assembled object into the paint bath and withdraw it before the downstream packaging step. In the digital model, this is a process-transfer sequence; the supplied files do not provide evidence of simulated fluid behaviour, coating thickness, drainage, curing or drying time. An earlier presentation slide refers to spraying [R2, slide 4], but the team has clarified that immersion is the correct method.

Packaging

The final stage is to place the finished part in a box. In digital terms, this requires a valid product at the transfer location, a box or packaging target, and a means of representing the placement. The source materials explain the intended action but do not give a dimensioned box, product count per box or a verifiable packing trajectory.

Stage	What is supported	What is not demonstrated
Welding	Intended joining after superposition	Weld physics, weld quality, cycle time
Painting	Immersion in a paint bath, as confirmed by the team	Immersion depth, coating coverage, drying time, fluid behaviour
Packaging	Product placed in a box in scenario	Box geometry, packing sequence evidence

3 • TECHNICAL SPECIFICATIONS

3.5 Smart Components and software architecture

Component roles come directly from the initial project report [R1].

The team selected an event-driven control structure using RobotStudio Smart Components. Each block has a restricted responsibility and communicates with other elements using signals. Rather than making one complex object manage the complete line, this design separates product generation, movement, position detection, attachment, control decisions and object removal.

Component	Role in the project	Interaction with the rest of the line
Source	Create virtual incoming parts	Provides an object to the transport system
LinearMover	Move parts along a linear route	Starts, stops or continues as logic permits
PlaneSensor	Detect object crossing a defined plane	Generates a signal associated with an arrival event
Controller	Coordinate the next intended operation	Receives signals and determines subsequent actions
Attacher	Attach one digital object to another	Represents assembly or grasp association
Detacher	Release an attached digital object	Returns object to an independent state
Sink	Remove an object from the simulation	Represents end of the material-flow chain

The benefit of this modularity is interpretability. A problem in an upstream input generation can be distinguished conceptually from a transport or sensing issue, and the corresponding signal can be inspected. However, modularity alone does not guarantee a correct sequence: input conditions, timing and object reference frames must still be configured consistently.

3 • TECHNICAL SPECIFICATIONS

3.6 Functional signal chain and interlocks

The figure illustrates the chain described in [R1] and [R2]; it is not a screenshot of the actual logic editor.

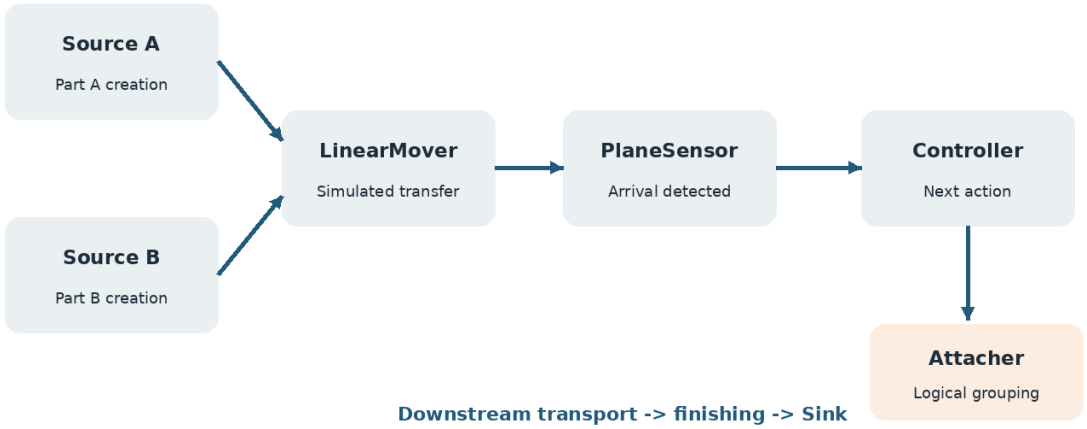


Figure 3. Simplified signal-driven architecture reconstructed from the project narrative.

At the start of a cycle, a Source generates an object. A LinearMover transports it until a PlaneSensor indicates that the object has reached a relevant position. The detection event becomes an input for the station logic. The Controller then coordinates what follows: a robot or assembly operation may be triggered, objects may be attached or released, and further movement is authorised. The downstream transport continues to finishing and packaging, with the final Sink marking the end of the virtual flow.

Control conditions that should be verified

- A detection signal should correspond to the correct component and the correct station.
- A new motion should not begin before the preceding operation is complete.
- Assembly should only be authorised when both input components are present.
- A finished-product event should have a unique and observable end condition.

The four conditions above are an engineering interpretation of a safe sequential process, not a verified list of interlocks in the current RobotStudio file. The original reports do not include the complete connection graph or the signal names.

3.7 Parameters: object creation, transport and detection

Based on the parameter descriptions and role definitions in [R1].

Configuring a Source

The initial report explicitly identifies five main Source fields. “Source” refers to the original virtual object, “Copy” to the generated object, “Parent” to its location in the station hierarchy, and “Position” and “Orientation” to the initial pose. An incorrect pose can create an object outside the intended conveyor path. An incorrect parent can also lead to unexpected behaviour when multiple simulated objects are regenerated.

Field	Purpose in the model	Recommended verification
Source	Prototype object to instantiate	Correct component geometry referenced
Copy	Generated scene object	A new object exists per intended event
Parent	Hierarchy/reference frame	Object is in the expected coordinate system
Position	Initial position of generated part	Matches infeed start point
Orientation	Initial object rotation	Part alignment matches transport direction

Configuring a LinearMover and PlaneSensor

The team used LinearMovers instead of simulated physical belt motion. The movement is therefore a defined translation of objects, rather than a mechanics-based conveyor model. PlaneSensors detect crossings at selected positions. Their results are converted into signals for the control sequence. The interaction should be checked with a single generated part before attempting the complete line, because a missed detection at an early stage can propagate as a failure later in the cycle.

The source materials do not contain numerical speed settings, detection-plane positions, delays or the sensor-to-controller signal names. These settings need to be extracted from the original RobotStudio project if exact reproducibility is required.

3 • TECHNICAL SPECIFICATIONS

3.8 Assembly logic, robot programs and traceability

The documentation reports component roles but contains no actual RAPID listing.

Assembly and release logic

Attacher and Detacher are used to represent virtual object relationships. Attacher allows one object to be fixed to another, which is useful either when a gripper holds a part or when two components must remain together. Detacher performs the reverse action. Their use has to be coordinated with robot motion: attaching a part before a successful pick or releasing it before reaching the target can break the material-flow sequence.

Robot motion

Robot motion programming in RobotStudio can involve joint and linear movements, as well as signal-based synchronisation. The project presentation mentions MoveJ, MoveL, SetDO and WaitDI as examples [R2, slide 11], but does not reproduce the implemented RAPID program. Accordingly, the assessment focuses on the required motion and control functions rather than on unverified program instructions.

Minimum information needed to verify the robot programs

Evidence item	Why it is needed	Current status
Robot model and tool definition	Checks reach, payload and reference frames	Not supplied
Pick and place targets	Confirms actual trajectory and alignment	Not supplied
Program excerpt or task tree	Confirms implemented motion instructions	Not supplied
Signal-to-program mapping	Explains handshake between motion and logic	Not supplied
Executed sequence recording	Shows the program ran as intended	Not supplied

Exact program verification requires the station name, robot task, target sequence, signal mapping and corresponding RAPID routine. The project documents do not contain sufficient information to reproduce that code reliably.

3.9 Functional requirements and acceptance criteria

Requirement matrix derived from the documented process; acceptance methods are proposed.

ID	Functional requirement	Suggested evidence / acceptance
FR-01	Generate both incoming components	Observe exactly one part per intended Source event
FR-02	Carry components to pick positions	Watch each LinearMover reach its intended endpoint
FR-03	Detect arriving parts	Check each relevant PlaneSensor output changes
FR-04	Position both parts together	Review robot targets and final relative pose
FR-05	Represent the joining stage	Confirm the assembly association is retained
FR-06	Represent coating	Confirm selected technique and station completion event
FR-07	Place product in packaging	Verify object reaches box target
FR-08	Remove completed product	Check Sink reaches an expected end state
FR-09	Maintain cycle repeatability	Re-run from a defined reset and compare outcomes
FR-10	Document the final implementation	Supply code, signals, screenshots and test record

The original assignment establishes the required overall outcome, but not numeric tolerance limits for these requirements. To turn the model into a verifiable engineering deliverable, each requirement should be linked to an observable signal, visible object state, recorded image or code excerpt. The proposed acceptance points above intentionally avoid claiming an unmeasured cycle time or success percentage.

The requirement IDs allow test records to link each operating condition to an expected observation and a decision to accept the behaviour or investigate an anomaly. This creates a direct relationship between the specification and the demonstration.

4 • PROBLEMS AND CONSTRAINTS

4.1 Computing-resource and software-stability problems

The team explicitly reported instability when the computer was not powerful enough [R1].

The first observed difficulty was execution reliability. According to the original team report, the software could produce bugs when run on a machine with insufficient computing resources. In a complex virtual scene, several robots, animated objects, geometry instances, sensors and control links can require simultaneous updates. A resource-related interruption may therefore appear at the level of movement or logic even when the intended sequence is understandable.

The evidence does not indicate which resource was limiting, whether CPU, RAM or graphics performance was the principal factor, nor which hardware configuration was tested. It would consequently be incorrect to claim a specific bottleneck or minimum workstation specification. The robust conclusion is that computing capability influenced the stability of the team's simulation experience.

Potential consequences within the planned workflow

- The scene may execute incompletely or appear to stop during a multi-stage sequence.
- An expected movement may not complete before the next control event is evaluated.
- It becomes harder to distinguish a faulty signal relationship from an execution problem.
- Repeated troubleshooting can consume project time without establishing a stable baseline.

What should be logged next

A future debug log should include the date, computer used, RobotStudio version, active station, exact stage where the failure appeared, any error message, and whether the same failure occurred after a clean reset. These entries are proposed documentation practices; the current source files do not include such a log.

4 • PROBLEMS AND CONSTRAINTS

4.2 Repeatability and cumulative failures

Both difficulties appear explicitly in the source report's problems section [R1].

Repeatability

The team observed that the same apparently unchanged station logic could lead to different simulation results. This made it difficult to determine whether a modification was necessary and whether a previous correction had been effective. Repeatability is especially important in a signal-driven model, because the position or state of an object at the moment a sensor is evaluated can influence subsequent transitions.

Accumulated errors

The report also states that bugs could accumulate and affect the final outcome. This can be interpreted as a propagation problem: if an object is not created, moved or detected as expected, later operations do not receive the state on which they depend. A downstream failure may therefore originate in an earlier station. The documents do not identify a single confirmed root cause for every malfunction.

Reported symptom	Plausible explanation (not confirmed)	Diagnostic check
Different outcomes with same logic	Initial state or event timing differs	Reset all states before rerun
Part not seen downstream	Missed or mistimed detection	Observe upstream sensor transitions
Unexpected object association	Attachment/release sequence differs	Inspect object hierarchy and active links
Later stage fails	Earlier transition did not complete	Trace backwards from last correct event

These explanations are hypotheses for future troubleshooting, not established project findings. A controlled experiment would be necessary to distinguish timing, resource consumption, object state and configuration issues.

5 • SOLUTIONS AND DESIGN CHOICES

5.1 Simplifying the material-flow model

The use of LinearMovers instead of mechanical conveyors is explicitly described in [R1].

The main documented modelling decision is the replacement of real conveyor mechanics with LinearMover components. Instead of simulating a conveyor's belt, drive and contact interaction, the system commands a translated object along a defined path. This supports the assignment's immediate objective of coordinating successive manufacturing operations while reducing the number of mechanical details represented in the scene.

This simplification should not be confused with a fully validated solution to the software difficulties. The report confirms that the team used LinearMovers, but it does not demonstrate a before-and-after stability comparison. It is best presented as a design choice that makes the material-flow architecture more manageable, with performance benefits remaining unquantified.

Trade-off analysis

Aspect	LinearMover-based approach	Physical conveyor simulation
Complexity	Lower: move object along a specified path	Higher: movement through mechanics/contact
Control integration	Straightforward signal start and stop	Potentially more realistic control interfaces
Geometry detail	Does not require a driven belt	Includes transport mechanism
Process fidelity	Limited representation of slip/collisions	Potentially richer physical behaviour
Use in this project	Documented implementation choice	Possible future development

The decision is appropriate for an introductory digital-manufacturing exercise as long as the abstraction is clearly explained. The purpose is to simulate when a part moves and arrives, not to predict the mechanical efficiency of the conveyor system.

5 • SOLUTIONS AND DESIGN CHOICES

5.2 Modular coordination and proposed robustness measures

Architecture is documented; the validation controls below are recommendations.

The second central design choice is the use of separate Smart Components and signal connections. A Source only creates a part, a PlaneSensor only reports a detection, and an Attacher only manages an association. Keeping these functions conceptually independent makes it easier to isolate an unexpected behaviour than when the full line is treated as a single opaque process.

Recommended control principles

10. Establish a known start state in which previous parts and stale output signals have been cleared.
11. Test a single component and one sensor before linking multiple stations.
12. Make each transition depend on an observable completion condition, not only on an assumed delay.
13. Ensure assembly is authorised only after both required input parts are present.
14. Record failure location and object state before changing several connections at once.

These principles are not listed as verified fixes in the initial report. They form a practical improvement plan based on the documented signal architecture and the observed repeatability difficulty.

A simple diagnostic hierarchy

Diagnostic level	Question
Object creation	Did the right component appear in the expected frame?
Transport	Was the component moved to the correct position?
Detection	Did the sensor signal reflect the object's arrival?
Control	Was the expected action selected at the right time?
Robot and assembly	Did the intended positioning and attachment occur?
Downstream result	Did the completed object continue to the next stage?

6 • IMPLEMENTATION PROCESS

6.1 From functional idea to virtual station

Chronological synthesis inferred from the team's presentation and reported approach [R1, R2].

Step 1 — Define the manufacturing scenario

The team first specified the transformation to simulate. Two input objects must be brought together and subsequently treated as one product. The chosen downstream processes were welding, painting and packaging. This first step was necessary because it defined the work areas, required material flows and interaction points needed in the 3D environment.

Step 2 — Build the RobotStudio environment

The station was then assembled as a virtual manufacturing scene. The original report shows an isometric overview of robots, conveyors and parts. The presentation describes the use of ABB RobotStudio library objects and the possibility of importing geometry from CAD through the Import Geometry function, but it does not establish the exact imported part list or export file formats. These details remain to be documented from the original working station.

Step 3 — Identify automation events

For each movement or transformation, the model needs an event that makes the next operation meaningful. A part is created, arrives at a station, is handled by a robot or logically attached, and then leaves for another station. This decomposition allows a control block to respond to a specific state rather than a visual impression of where the product is.

Step 4 — Connect and simulate

The station logic connects the generated objects, movements and sensors. The documented operating sequence runs from Source to LinearMover, PlaneSensor, Controller, Attacher, additional transport and Sink. The resulting simulation was subject to the stability and repeatability problems already described; a complete and consistently successful run is not independently established by the supplied evidence.

6 • IMPLEMENTATION PROCESS

6.2 Building the station logic incrementally

Detailed implementation rationale based on the reported Smart Component set [R1].

The implementation is easiest to understand as a set of small functions joined into a production chain. Initially, virtual components must be made available in the scene. Next, linear transport functions move those components toward the assembly region. Once a detection condition is reached, the logic signals the next station. The association between objects is managed through attachment functions. Finally, a completed object is advanced and removed from the environment.

Component-level checks during integration

Order	Component configuration	Visual or logical check
1	Place Source and define instance prototype	Part appears at intended start position
2	Define LinearMover path	Object follows expected route
3	Place PlaneSensor	Sensor output changes at crossing
4	Connect sensor signal to control	Next action reacts to real event
5	Configure Attacher / Detacher	Object relationship changes at intended step
6	Connect downstream movements	Assembly leaves without duplicating parts
7	Add Sink at terminal condition	Completed object exits simulation

These checks explain a sensible integration strategy; the team's files do not contain a dated step-by-step build log. In the actual station, some operations may have been configured in a different order. The important engineering principle is that adding one uncertain behaviour at a time makes errors easier to localise.

The Source properties, sensor connections and Attacher configuration are key points to inspect when checking the integration of the station logic. They show whether the expected signal relationships have been implemented.

6 • IMPLEMENTATION PROCESS

6.3 Robot motion and workstation interactions

Evidence gaps are retained rather than replaced with fictional robot paths or code.

Robot motion is the physical-looking part of the virtual sequence. For pick and place, the robot needs an approach pose, a pick pose, a departure pose, and a target pose for assembly. The motion also needs to be coordinated with attachment and release of the component. Comparable target and event definitions would be required at the later stations if their processing is robotically executed.

The robot-motion section of the project presentation identifies RAPID programs as a key part of the system but does not show an executable code extract [R2, slide 11]. Consequently, the precise target order, motion speed, tool configuration and program names cannot be verified from the available project records.

Interfaces that must agree

Motion function	Input condition	Required confirmation
Pick first part	Part A available at pick location	Attachment / grasp established
Pick second part	Part B available at its location	Attachment / grasp established
Place / superpose	Assembly target available	Correct relative pose at target
Release product	Transfer position reached	Part no longer follows gripper
Next process stage	Input product arrives	Station completion / transfer event

The table summarises the necessary interface conditions. A full engineering verification would additionally require the robot target list, tool-centre-point definitions and signal handshake recorded in the RobotStudio project.

6 • IMPLEMENTATION PROCESS

6.4 Simulation execution, debugging and iteration

The team's reported instability makes repeat-run methodology particularly relevant.

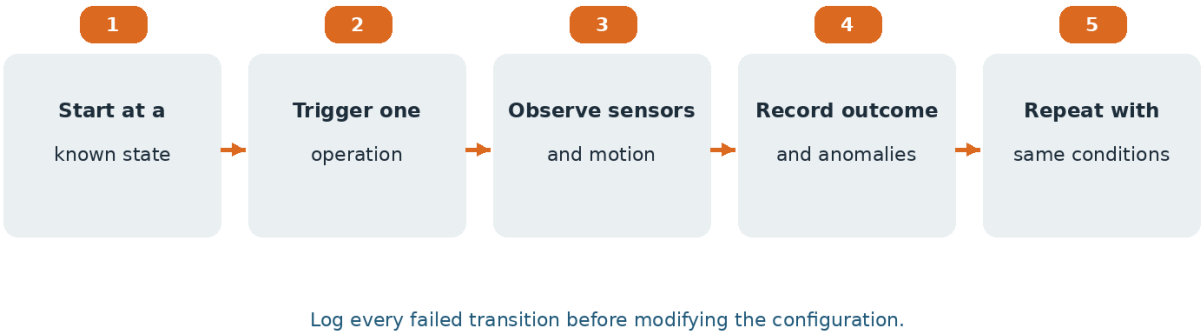


Figure 4. Proposed test iteration loop; this is a recommendation for future verification.

An end-to-end demonstration should not be the first test of a multi-station model. Once a basic object can be generated and detected, each additional transformation can be verified separately, then combined. After a change, a fresh simulation should be launched from the same initial state so that different outcomes can be compared meaningfully.

Suggested debugging procedure

15. Reset the station to a predefined state and record the active configuration.
16. Run only the first infeed and verify generation, movement and detection.
17. Enable the assembly action and check the state of both objects after placement.
18. Add the joining, coating and packaging transitions in sequence.
19. Repeat the full cycle while logging the last confirmed event whenever an error occurs.

This procedure directly addresses the team's experience that unchanged logic could lead to varying results. It would also help locate accumulated failures without changing several potential causes simultaneously. Since the provided documents do not give a recorded trial series, the procedure is a proposed corrective method rather than a retrospective claim that all issues were solved.

7 • RESULTS

7.1 Results supported by the available evidence

Summary of the implemented model and qualitative evaluation.

The principal tangible result is a model of a four-operation automated production line within ABB RobotStudio. The initial report includes a rendered overview of the station. It also documents a functional logic design using Sources, LinearMovers, PlaneSensors, a Controller, attachment-related components and a Sink. The presentation translates the same concept into a clear production sequence and explains the intended signal chain.

Evidence audit

Claim	Evidence available	Status
A four-step production scenario was defined	[R1] steps; [R2] slide 4	Documented
A virtual station was created	Actual station screenshot in [R1]	Documented
Station Smart Components were planned/described	Descriptions in [R1] and [R2]	Documented concept
Conveyors are abstracted by LinearMovers	Explicit statement in [R1]	Documented design choice
A complete robot code was supplied	No code listing; placeholders in [R2]	Not demonstrated
A full cycle was recorded and verified	Slide [R2] requests video/screenshots	Not demonstrated
Throughput and cycle time were quantified	No numerical results in either source	Not available
Simulation was consistently repeatable	Problems section describes the opposite	Not established

This audit supports a cautious interpretation of the outcome. The virtual environment and its logical architecture were developed and documented, but the evidence package is not yet sufficient to validate every production transition. The value of the work is therefore in the defined system and the design lessons it produced, rather than in a claimed industrial production result.

7 • RESULTS

7.2 Discussion: achievements, limitations and reliability

Interpretive analysis grounded in the documented system and the reported errors.

What the project demonstrates

The project shows how a production chain can be represented through a combination of 3D objects and interconnected control blocks. It also identifies the importance of material flow when two separate inputs become one combined product. The students documented the roles of the main components and produced a logical description of their interactions, demonstrating an understanding of the relationship between simulation objects and station behaviour.

Why a successful demonstration cannot be assumed

The major limitation is reliability. The original report reports bugs related to computer capability, inconsistent outcomes for apparently unchanged logic and failures that accumulate. In the absence of a repeat-run record, these statements preclude treating the model as a stable production system. Even if a successful cycle was observed by the team, the provided documents do not establish how often success occurred, under which conditions or with what configuration.

Other limits on interpretation

- No documented robot model or tool parameters are available for checking kinematic feasibility.
- The coating operation is confirmed as immersion in a paint bath, but the supplied evidence does not quantify immersion time, coating coverage or drying behaviour.
- No exact timing, throughput, failure-rate or energy data are provided.
- No full RAPID code, signal map or video of the entire simulated sequence appears in the files.

These limitations are not grounds for inventing figures. They identify the most useful evidence to collect before claiming successful validation of the line.

7 • RESULTS

7.3 Proposed validation methodology

Procedure proposed to evaluate simulation repeatability and sequence correctness.

Test	Action	Expected observation	Record
T01	Start from reset; create one Part A	Correct object at intended infeed	Screenshot + event time
T02	Move Part A past its PlaneSensor	Movement and sensor activation	Signal log
T03	Repeat for Part B	Second input follows intended route	Signal log
T04	Trigger pick-and-place logic	Two parts are superposed	Target screenshot
T05	Trigger joining association	Parts remain grouped	Object tree + scene
T06	Run coating transfer	Product visits painting region	Station screenshot
T07	Run packaging transfer	Product reaches box	Station screenshot
T08	Complete cycle to Sink	Product reaches end condition	Completion log
T09	Repeat unchanged cycle multiple times	Outcomes are consistent	Run-by-run pass/fail list
T10	Restart from clean state	No stale objects or signals remain	Reset record

For each test, a test sheet should specify the project version, initial conditions, actual outcome, anomaly if any, and a screenshot or log identifier. After a change, the same test should be repeated rather than replacing it with an informal demonstration. A stable test record is more persuasive than a single attractive 3D image.

Numerical performance: not yet available

Throughput could eventually be reported as completed products per unit time and cycle duration as elapsed time between two comparable production events. No measurements are supplied, so this report intentionally leaves both values unreported. The first performance goal should be repeatability; only then would optimisation of cycle time be meaningful.

8 • CONCLUSIONS AND PERSPECTIVES

8.1 Conclusions

Summary of deliverables, learning value, unresolved issues and evidence status.

The team designed a virtual smart-factory scenario that transforms two incoming components into a single finished and packaged product by passing through pick and place, welding, painting and packaging. ABB RobotStudio was used to represent the production layout and to organise its behaviour around Smart Components and signal links. In the documented architecture, Sources generate components, LinearMovers represent the conveyor transport, PlaneSensors detect position events, the Controller coordinates actions, Attacher/Detacher manage object associations, and the Sink removes the final object.

The project's central technical lesson is that a production line must be coordinated as a system rather than as a collection of separate animations. Each stage depends on an upstream state: successful generation enables transport, arrival detection enables the next action, and correct assembly enables downstream finishing. A failure in one event can therefore affect several later stations.

The team encountered three important difficulties: software instability on an insufficiently capable computer, inconsistent simulation outcomes under apparently unchanged logic, and the propagation of accumulated errors. These observations showed the need to combine graphical modelling with disciplined debugging and repeatable testing.

The available evidence supports the existence of the scenario, station model and described logic, but does not establish a measured, fully repeatable end-to-end production cycle. The project should therefore be presented as a developed virtual manufacturing model with a clear validation agenda, not as a certified industrial solution.

8 • CONCLUSIONS AND PERSPECTIVES

8.2 Perspectives and development roadmap

All items below are proposed next steps and should not be read as completed results.

Priority	Next activity	Purpose / expected evidence
Immediate	Verify immersion-station setup	Confirm paint bath placement and the dipping trajectory
Immediate	Export actual RobotStudio screenshots	Prove station setup, logic and robot paths
Immediate	Create a repeatable test protocol	Document pass/fail for the same conditions
High	Record software and hardware versions	Make stability observations reproducible
High	Document robot models and target frames	Check manipulation and geometric feasibility
High	Build complete signal/IO list	Make event dependencies auditable
Medium	Measure time for repeated stable cycles	Establish a supported throughput baseline
Medium	Review collisions and downstream buffers	Improve feasibility and robustness
Longer term	Replace simplified conveyor abstraction	Assess higher-fidelity transport behaviour
Longer term	Explore PLC communication	Study connection with industrial automation

The order of improvement matters. Before adding more realism, the current system should be made understandable and consistently executable. A well-documented baseline can reveal whether a future change improves the result or merely adds complexity. Once repeated cycles succeed under the same conditions, more precise robot targets, conveyor behaviour and throughput measurements can support an engineering assessment.

The longer-term objective is a repeatable end-to-end simulation supported by traceable station logic, verified robot motions and a documented production cycle. Such a baseline would make it possible to compare alternative line configurations and assess practical improvements before moving toward a physical implementation.